

Nginx와 Apache

웹 서버를 선택할 때 자주 비교되는 Apache와 Nginx의 동작 방식과 차이를 정리해보았습니다
두 서버는 요청을 처리하는 구조가 다르며, 이 차이가 성능 특성과 적합한 사용 환경을 결정합니다

1. Apache

Apache는 **Multi-Processing Module(MPM)** 방식으로 요청을 처리하며, Prefork, Worker, Event 세 가지 모드를 선택할 수 있다.

MPM은 Apache가 네트워크 포트를 바인딩하고 요청을 받아들이는 방식을 결정하는 핵심 모듈이다.

1.1 Prefork

- 요청 하나당 프로세스 하나를 할당한다
- 미리 자식 프로세스를 띄워놓고 요청이 오면 분배한다
- 동시 접속이 1000개라면 프로세스도 약 1000개가 된다
- 동시 접속이 많을수록 메모리 사용량이 올라간다

1.2 Worker

- 프로세스 + 스레드 하이브리드 방식이다
- 프로세스가 여러 개이며, 각 프로세스 안에 스레드가 여러 개 존재한다
- 하나의 스레드가 하나의 연결을 담당한다
- Prefork보다 메모리 효율이 더 좋다

1.3 Event

- Worker 구조에 더해, Keep-Alive 연결을 별도의 전용 스레드가 관리한다
- 기존 Worker 방식에서는 Keep-Alive 대기 중인 연결이 스레드를 점유하는 문제가 있었는데, Event는 이를 분리해 해결한다
- Nginx의 이벤트 기반 처리 방식을 일부 차용한 모델이다

1.4 장점

- .htaccess 로 디렉토리별 세밀한 제어가 가능하다
- 오랜 역사로 참고 자료와 모듈 생태계가 풍부하다
- mod_php 등으로 동적 콘텐츠를 서버 내부에서 직접 처리할 수 있다

1.5 단점

- 동시 접속자가 증가하면 CPU / Memory 부담이 급증한다 (특히 Prefork)

- 정적 파일 서빙 성능이 Nginx에 비해 떨어진다
- .htaccess 를 사용하면 요청마다 디렉토리를 탐색하므로 성능 저하 요인이 될 수 있다

1.6 적합한 경우

- 공유 호스팅 환경 (사용자별 .htaccess 필요)
- 레거시 PHP 시스템
- 트래픽이 크지 않고 안정성과 익숙함이 우선인 경우
- 특정 Apache 모듈 의존도가 높은 경우

2. Nginx

Nginx는 **Event-driven, Asynchronous, Non-Blocking** 구조를 기반으로 한다.

- 마스터 프로세스 1개와 워커 프로세스 N개로 구성된다 (워커 수는 보통 CPU 코어 수에 맞춘다)
- 각 워커 프로세스가 수천 개의 연결을 동시에 처리한다
- 설정은 중앙 집중식이며, Apache의 .htaccess 처럼 디렉토리에 둔 파일로 동적으로 설정을 바꾸는 방식은 지원하지 않는다 (단, location 블록을 통한 경로별 설정은 가능하다)
- 동적 콘텐츠를 자체적으로 실행하지 않으며, FastCGI·uWSGI·프록시 등을 통해 백엔드에 위임한다

Request 1, 2, ... N -----> Worker 1 -----> Worker 2 ...

각 워커가 이벤트 루프를 돌러가며 처리한다

이벤트 루프 안에서 OS 레벨의 비동기 I/O를 사용한다.

I/O 작업이 끝나면 OS가 워커에게 완료를 알려주고, 워커는 그 시점에 결과를 처리한다.

이런 방식으로 워커 하나가 수많은 연결을 블로킹 없이 처리한다.

2.1 장점

- 적은 리소스로 높은 동시성을 처리한다
- 정적 파일 서빙, 리버스 프록시, 로드 밸런서 용도로 적합하다
- 설정 파일 구조가 직관적이다

2.2 단점

- 동적 콘텐츠를 직접 처리하지 못하고 항상 백엔드에 위임한다
- .htaccess 방식의 디렉토리 단위 분산 설정을 지원하지 않는다 (모든 설정은 중앙 설정 파일에서 관리)

2.3 적합한 경우

- 동시 접속이 많은 경우
- 정적 파일 위주의 서비스
- 애플리케이션 서버 앞단의 리버스 프록시

3. Nginx를 왜 앞에 둘까?

Nginx를 백엔드 애플리케이션 앞단에 두는 구성은 매우 일반적이다. 그 이유를 정리하면 다음과 같다.

3.1 80 / 443 포트 바인딩

- 리눅스에서 1024 미만 포트는 root 권한이 필요하다
- 백엔드를 root 권한으로 직접 띄우는 것은 보안상 바람직하지 않다
- Nginx가 80 / 443 포트를 받고, 내부적으로 백엔드가 사용하는 포트에 프록시한다

3.2 SSL / TLS 종료(Termination)

- HTTPS 인증서 관리와 암호화·복호화 연산은 비용이 크다
- Nginx에서 SSL/TLS를 종료(termination)하고, 내부망은 HTTP로 통신하도록 구성할 수 있다
- 인증서 갱신도 Nginx 레이어에서만 관리하면 되므로 운영이 단순해진다

3.3 정적 리소스 분리

- 정적 파일은 Nginx가 직접 서빙한다
- 백엔드는 비즈니스 로직 처리에만 집중할 수 있다

3.4 로드 밸런싱

- 백엔드를 여러 개 띄우고 Nginx가 트래픽을 분산한다
- 헬스 체크를 통해 비정상 백엔드를 제외할 수도 있다

정리

구분	Apache	Nginx
처리 방식	프로세스/스레드 기반 (MPM)	이벤트 기반 비동기
동시성	연결당 자원 소모 큼	적은 자원으로 높은 동시성
동적 콘텐츠	내부 처리 가능 (mod_php 등)	백엔드 위임
디렉토리별 설정	.htaccess 지원	중앙 설정만 지원
정적 파일	상대적으로 느림	빠름

실무에서는 **Nginx**를 앞단(리버스 프록시)에 두고, 뒤에 애플리케이션 서버(또는 **Apache**)를 두는 조합이 널리 쓰인다. 각 서버의 강점을 살린 구성이라 할 수 있다.